

01 Embodied AI Applications in Real-Time Detection

1. Brief Game Instructions

In this lesson, we will learn how to leverage a large model to analyze real-world scenes in real time and implement an interactive real-time detection activity. Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode. Once the program starts running and WonderEcho Pro announces "I am ready."

Say the wake-up word (the wake-up word depends on the firmware flashed, and by default, the wake-up word for our factory firmware is "Hello, HiWonder") to activate WonderEcho Pro. It will respond with "I am here."

By running this activity, we can freely express commands, allowing MentorPi to recognize the current scene, analyze it, and verbally report the analysis results.

2. Preparation

2.1 Verifying the WonderEcho Pro Firmware

By default, the wake-up word for the WonderEcho Pro factory firmware is "Hello, Hiwonder." If you wish to change the wake-up word to "MentorPi," please refer to the tutorial titled "20.1 Voice Module Overview and Installation\ 03 Firmware Flashing Method" for step-by-step instructions. If you have previously flashed the device with different firmware—such as the version with the wake word “小幻小幻” or “mentorpi”—you must use

the corresponding wake word to activate the device.

For example, if the “小幻小幻” firmware was installed, the correct wake word would be “小幻小幻”; if the “mentorpi” firmware was installed, the wake word would be “mentorpi.”

In the following tutorial, we will use the default factory wake word “Hello, HiWonder” for demonstration purposes.

2.2 Obtaining and Configuring the Large Model API Key

By default, the program does not include the configuration of the Large AI Model-related API keys. Before activating features related to the Large AI Model, please refer to the section "02 Multimodal Large Model Applications > 01 Obtaining and Configuring Large Model API Keys" to configure the necessary keys. This step is mandatory and cannot be skipped, as it is crucial for the proper functioning and experience of the large model features.

2.3 Network Configuration

Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode.

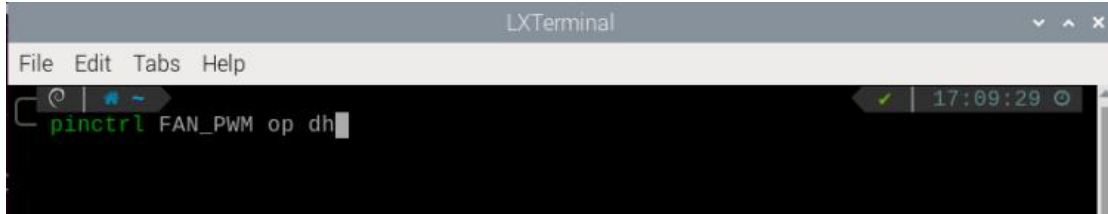
The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet. For detailed instructions on network configuration, please refer to the course “20 Network Configuration Courses\ 01 Network Configuration Instructions”

2.4 Turn Off Cooling Fan

- 1) Click  in the top-left corner to open the terminal.
- 2) Type the command and press Enter to turn off the Raspberry Pi's

cooling fan, minimizing noise interference during recording.

```
pinctrl FAN_PWM op dh
```



3. Starting and Stopping the Game

-
- ① The input commands must strictly observe case sensitivity and spacing.
 - ② The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet.
-

- 1) Start the robot and follow the instructions in “8. Remote Tool Installation Connection → 8.4 Docker Container Introduction and Entry” to connect via the VNC remote desktop software. Then, switch to the ROS2 environment and select the appropriate language version as needed.
- 2) Click the icon in the top-left corner to open the terminal and enter the ROS2 development environment.
- 3) Then, input the command to disable the auto-start service:

```
~/./stop_ros.sh
```

```
~/./stop_ros.sh  
zsh: killed ~/./stop_ros.sh
```

- 4) Enter the command to launch the corresponding game.

```
ros2 launch large_models vllm_with_camera.launch.py
```

```
ros2 launch large_models vllm_with_camera.launch.py
```

- Once the command line outputs the message below and the device announces "I am ready," it indicates that the voice device has been initialized. You can now say the wake word, "hello Hiwonder."

```
[agent_process-10] [INFO] [1742291623.374386995] [agent_process]: start
[tts_node-11] [INFO] [1742291624.176498371] [tts_node]: start
[vocal_detect-9] [INFO] [1742291624.184420787] [vocal_detect]: start
[ascamera_node-6] 2025-03-18 09:53:44[INFO] [CameraHp60c.cpp] [260] [startStreaming] start streaming
[agent_process-10] [INFO] [1742291624.996223930] [agent_process]: set model
[vllm_with_camera-12] [INFO] [1742291625.255926774] [vllm_with_camera]: start
```

- When the following output appears in the command line, MentorPi will announce "I am Here," indicating that WonderEcho Pro has been successfully activated. At this point, the system will begin recording the user's command.

```
[vocal_detect-7] [INFO] [1741419606.799872983] [vocal_detect]: asr...
[vocal_detect-7] [INFO] [1741419606.805699617] [vocal_detect]: Recording.....
[vocal_detect-7] [INFO] [1741419608.378862834] [vocal_detect]:
```

- You can freely phrase your commands to control the movement of MentorPi, such as saying, "**Describe what you saw.**"
- When the command line displays the output shown below, it indicates that WonderEcho Pro has printed the recognized speech.

```
[vocal_detect-9] [INFO] [1747365662.570539600] [vocal_detect]: asr time: 1.16
[vocal_detect-9] [INFO] [1747365663.257929855] [vocal_detect]: publish asr result:Describe what you saw.
[vocal_detect-9]
```

- When the command line shows the output below, it indicates that the cloud-based large speech model's speech recognition service has successfully processed the user's command audio. The recognition result will be displayed under "publish_asr_result."

```
[vocal_detect-9] [INFO] [1747365663.257929855] [vocal_detect]: publish asr result:Describe what you saw.
[vocal_detect-9]
[agent_process-10] [INFO] [1747365663.258393982] [agent_process]: thinking...
[agent_process-10] [INFO] [1747365679.811942131] [agent_process]: publish vllm result:The image shows a workspace with various electronic components and tools. On the left side, there is a small electronic device with a label that reads "200mAh Lithium Battery." This device has red and black wires connected to it. In the background, there are some plastic bags and a monitor partially visible. On the right side, there is a transparent plastic container with some cables and possibly other small electronic parts inside. The container is placed on a white surface, and there are more cables and a black object in the foreground. The overall scene suggests a setup for electronics work or experimentation.
```

- When the command line displays the output shown below, it indicates

that the cloud-based large language model has been successfully called to process the user's command and provide a verbal response labeled as "response."

The response is automatically generated by the large model, ensuring the accuracy of the meaning, though the wording and structure of the reply may vary.

11) When the command line shows the output below, it indicates that one round of interaction has been completed. You can refer to Step 4) to reactivate the voice device using the wake-up word and begin a new round of interaction.

```
[tts_node-11] [INFO] [1747365711.811615532] [tts_node]: [Playback Thread] Playback stream stopped and closed.  
[tts_node-11] [INFO] [1747365711.819716049] [tts_node]: [TTS] Playback task complete.  
[vocal_detect-9] [INFO] [1747365711.830597573] [vocal_detect]: enable_wakeup
```

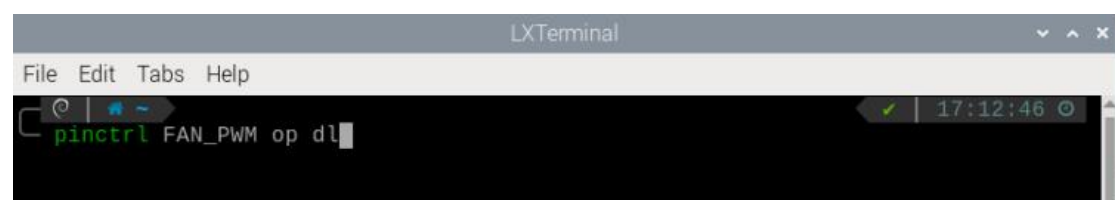
12) To exit this feature, simply press "Ctrl+C" in the terminal. If it doesn't close on the first attempt, you may need to press it multiple times.

```
[vocal_detect-5] [INFO] [1733998393.217872795] [vocal_detect]: enable_wakeup
```

13) Click the icon in the top-left corner to open the terminal.

14) Type the command and press Enter to turn on the Raspberry Pi cooling fan.

pinctrl FAN_PWM op dl



4. How It Works

Once the feature is activated, you can freely phrase your commands, such as saying, "**Describe what you saw**" MentorPi will automatically

recognize the scene within its field of view and provide a detailed description of the current surroundings.

5. Brief Program Analysis

5.1 Analysis of launch File

The program is saved at:

/home/ubuntu/ros2_ws/src/large_models/launch/vllm_with_camera.launch.py

◆ Library Files Import

```
1 import os
2 from ament_index_python.packages import get_package_share_directory
3
4 from launch_ros.actions import Node
5 from launch.substitutions import LaunchConfiguration
6 from launch import LaunchDescription, LaunchService
7 from launch.launch_description_sources import PythonLaunchDescriptionSource
8 from launch.actions import IncludeLaunchDescription, DeclareLaunchArgument, OpaqueFunction
```

- ① `ament_index_python.packages.get_package_share_directory`:
Used to obtain the shared directory path of a ROS2 package.
- ② `launch_ros.actions.Node`: Used to define a ROS2 node
- ③ `launch.substitutions.LaunchConfiguration`: Used to retrieve parameter configurations from a Launch file.
- ④ `LaunchDescription, LaunchService`: Used to define and launch a Launch file.
- ⑤ `launch_description_sources PythonLaunchDescriptionSource`:
Used to include other Launch files.
- ⑥ `launch.actions.IncludeLaunchDescription, DeclareLaunchArgument, OpaqueFunction`: Used to define actions and parameters within a Launch file.

◆ Define `launch_setup` Function

```

10 def launch_setup(context):
11     mode = LaunchConfiguration('mode', default=1)
12     mode_arg = DeclareLaunchArgument('mode', default_value=mode)
13
14     controller_package_path = get_package_share_directory('controller')
15     peripherals_package_path = get_package_share_directory('peripherals')
16     large_models_package_path = get_package_share_directory('large_models')
17
18     controller_launch = IncludeLaunchDescription(
19         PythonLaunchDescriptionSource(
20             os.path.join(controller_package_path, 'launch/controller.launch.py')),
21     )
22
23     depth_camera_launch = IncludeLaunchDescription(
24         PythonLaunchDescriptionSource(
25             os.path.join(peripherals_package_path, 'launch/depth_camera.launch.py')),
26     )
27
28     large_models_launch = IncludeLaunchDescription(
29         PythonLaunchDescriptionSource(
30             os.path.join(large_models_package_path, 'launch/start.launch.py')),
31         launch_arguments={ 'mode': mode }.items(),
32     )
33
34     vllm_with_camera_node = Node(
35         package='large_models',
36         executable='vllm_with_camera',
37         output='screen',
38     )
39
40     return [controller_launch,
41             depth_camera_launch,
42             large_models_launch,
43             vllm_with_camera_node,
44             ]

```

- ① `mode = LaunchConfiguration('mode', default=1)`: Defines a Launch parameter named `mode`, with a default value of 1.
- ② `sdk_launch`: Uses `IncludeLaunchDescription` to include the `MentorPi_sdk.launch.py` launch file from the `large_models` package, passing the `mode` parameter.
- ③ `large_models_launch`: Uses `IncludeLaunchDescription` to include the `start.launch.py` Launch file from the `large_models` package, passing the `mode` parameter.

◆ Define `generate_launch_description` function:

```

46 def generate_launch_description():
47     return LaunchDescription([
48         OpaqueFunction(function = launch_setup)
49     ])
50

```

◆ Main program entry:

```
51 if __name__ == '__main__':
52     # 创建一个LaunchDescription对象
53     ld = generate_launch_description()
54
55     ls = LaunchService()
56     ls.include_launch_description(ld)
57     ls.run()
```

- ① `ld = generate_launch_description()`: Generates a Launch description object.
- ② `ls = LaunchService()`: Creates a Launch service object.
- ③ `ls.include_launch_description(ld)`: Adds the Launch description to the Launch service.
- ④ `ls.run()`: Starts the Launch service, executing all the defined Launch actions.

5.2 Analysis of Python Files

The source code is located at:

/home/ubuntu/ros2_ws/src/large_models/large_models/vllm_with_camera.py

◆ Feature Package Import

```
5 import os
6 import cv2
7 import queue
8 import rclpy
9 import threading
10 import numpy as np
11 from cv_bridge import CVBridge
12 from rclpy.node import Node
13 from sensor_msgs.msg import Image
14 from std_msgs.msg import String, Bool
15 from std_srvs.srv import Trigger, SetBool, Empty
16 from rclpy.executors import MultiThreadedExecutor
17 from rclpy.callback_groups import ReentrantCallbackGroup
18
19 from speech import speech
20 from large_models.config import *
21 from large_models_msgs.srv import SetModel, SetString
```

- ① **cv2**: Used for image processing and display with OpenCV.
- ② **json**: Used for handling JSON-formatted data.
- ③ **sdk.fps as fps**: Used to calculate and display the frame rate

(FPS).

- ④ speech: Module related to large model voice interaction
- ⑤ large_models_msgs.srv: Custom service type for large models.
- ⑥ large_models.config: Configuration file for large models.

◆ VLLMWithCamera Class

```

24 class VLLMWithCamera(Node):
25     def __init__(self, name):
26         rclpy.init()
27         super().__init__(name)
28         self.image_queue = queue.Queue(maxsize=2)
29         self.vllm_result = ''
30         self.running = True
31         self.bridge = CvBridge()
32         timer_cb_group = ReentrantCallbackGroup()
33         self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
34         self.create_subscription(Image, 'ascamera/camera_publisher/rgb0/image', self.image_callback, 1)
35         self.create_subscription(String, 'agent_process/result', self.vllm_result_callback, 1)
36         self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_callback, 1, callback_group=timer_cb_group)
37         self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wakeup')
38         self.awake_client.wait_for_service()
39         self.set_model_client = self.create_client(SetModel, 'agent_process/set_model')
40         self.set_model_client.wait_for_service()
41         self.timer = self.create_timer(0.0, self.init_process, callback_group=timer_cb_group)

```

- ① self.vllm_result: Stores the result received from the agent_process/result topic.
- ② self.tts_text_pub: Creates a publisher to send text to the speech synthesis node.
- ③ self.create_subscription: Creates subscribers to receive image data, VLLM results, and speech playback completion signals.
- ④ self.awake_client and self.set_model_client: Create clients to call the wake-up and model setting services.

◆ get_node_state Method

```

43 def get_node_state(self, request, response):
44     return response

```

return response: Returns the response object.

◆ init_process Method

```

46     def init_process(self):
47         self.timer.cancel()
48
49         msg = SetModel.Request()
50         msg.model = aliyun_vllm_model
51         msg.model_type = 'vllm'
52         msg.api_key = aliyun_api_key
53         msg.base_url = aliyun_base_url
54         self.set_model_client.call_async(msg)
55         #self.send_request(self.set_model_client, msg)
56
57         speech.play_audio(start_audio_path)
58         threading.Thread(target=self.process, daemon=True).start()
59         self.create_service(Empty, '~/init_finish', self.get_node_state)
60         self.get_logger().info('\033[1;32m%s\033[0m' % 'start')

```

- ① SetModel.Request: Creates a request message to set the model.
- ② SetString.Request: Defines the request message for setting the prompt string.
- ③ set_pose_target: Sets the robot's initial pose.
- ④ Self.send_request: Sends a service request.
- ⑤ speech.play_audio: Plays an audio clip.
- ⑥ threading.Thread: Starts a new thread for image processing and tracking.

◆ Send_request Method

```

62     def send_request(self, client, msg):
63         future = client.call_async(msg)
64         while rclpy.ok():
65             if future.done() and future.result():
66                 return future.result()

```

- ① Client.call_async(msg): Asynchronously calls a service.
- ② future.done() and future.result(): Check if the service call has completed and retrieve the result.

◆ vllm_result_callback method:

```

68     def vllm_result_callback(self, msg):
69         self.vllm_result = msg.data

```

Receives results from the agent_process/result topic and stores them in self.vllm_result.

◆ process Method

```
71 def process(self):
72     cv2.namedWindow('image', 0)
73     cv2.moveWindow('image', 1920 - display_size[0], 0)
74     os.system("wmctrl -r image -b add,above")
75     cv2.waitKey(10)
76     while self.running:
77         image = self.image_queue.get(block=True)
78         if self.vllm_result:
79             msg = String()
80             msg.data = self.vllm_result
81             self.tts_text_pub.publish(msg)
82             self.vllm_result = ''
83
84         cv2.imshow('image', cv2.resize(image, (display_size[0], display_size[1])))
85         cv2.waitKey(1)
86     cv2.destroyAllWindows()
```

- ① self.image_queue.get: Retrieves image data from the queue.
- ② self.vllm_result: If there is a VLLM result, it is published to the speech synthesis node.
- ③ Cv2.imshow: Displays the image.

◆ Play_audio_finish_callback Method:

```
88 def play_audio_callback(self, msg):
89     if msg.data:
90         msg = SetBool.Request()
91         msg.data = True
92         self.send_request(self.awake_client, msg)
```

When the speech playback is complete, sends a request to enable the wake-up function.

◆ image_callback method

Converts received ROS image messages into NumPy arrays and stores them in the queue.

◆ Main Method

```
103 def main():
104     node = VLLMWithCamera('vllm_with_camera')
105     executor = MultiThreadedExecutor()
106     executor.add_node(node)
107     executor.spin()
108     node.destroy_node()
```

- ① Creates an instance of the VLLMWithCamera node.
- ② Uses a multithreaded executor to handle node tasks.
- ③ Calls executor.spin() to start processing ROS events.
- ④ Destroys the node when finished by calling node.destroy_node().